

Experiment No. 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

MetaMask is a **cryptocurrency wallet and gateway** to blockchain applications. It allows users to store, send, and receive Ether or tokens, and connect to decentralized applications (DApps) directly from a web browser. MetaMask acts as a **bridge** between a user's browser and the Ethereum blockchain (or other compatible networks).

Features:

- Easy management of multiple blockchain accounts
- Secure private key storage
- Integration with test networks like **Sepolia**, **Linea**, or **RSK**
- Interaction with DApps and smart contracts via **Remix IDE**

Testnet:

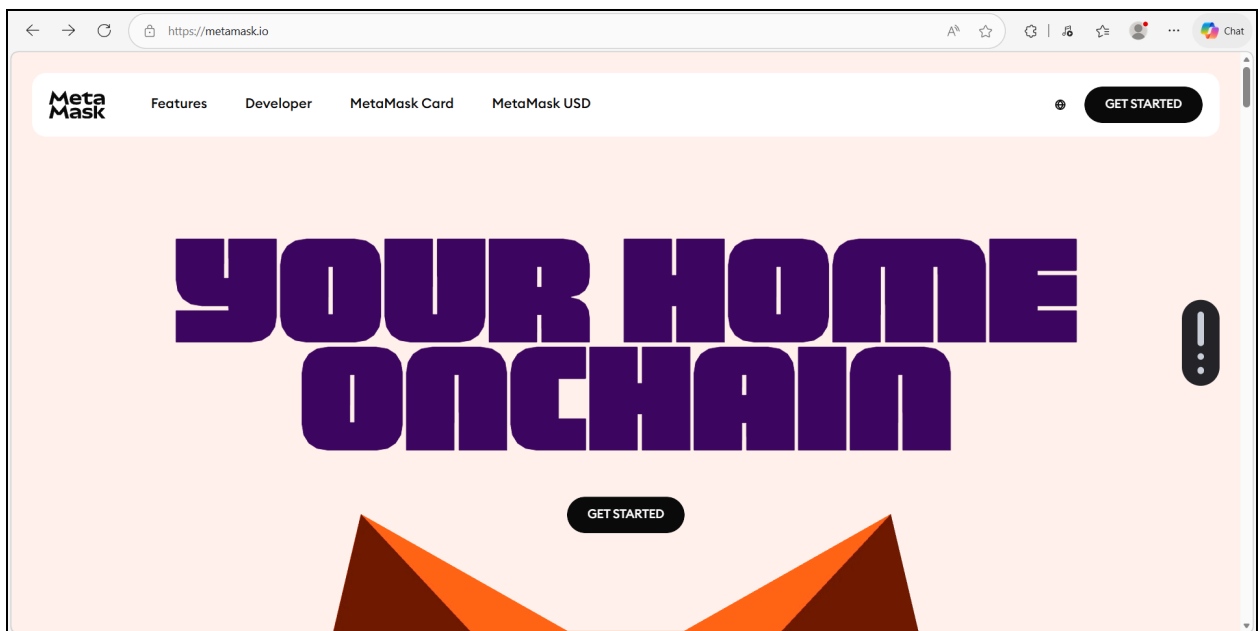
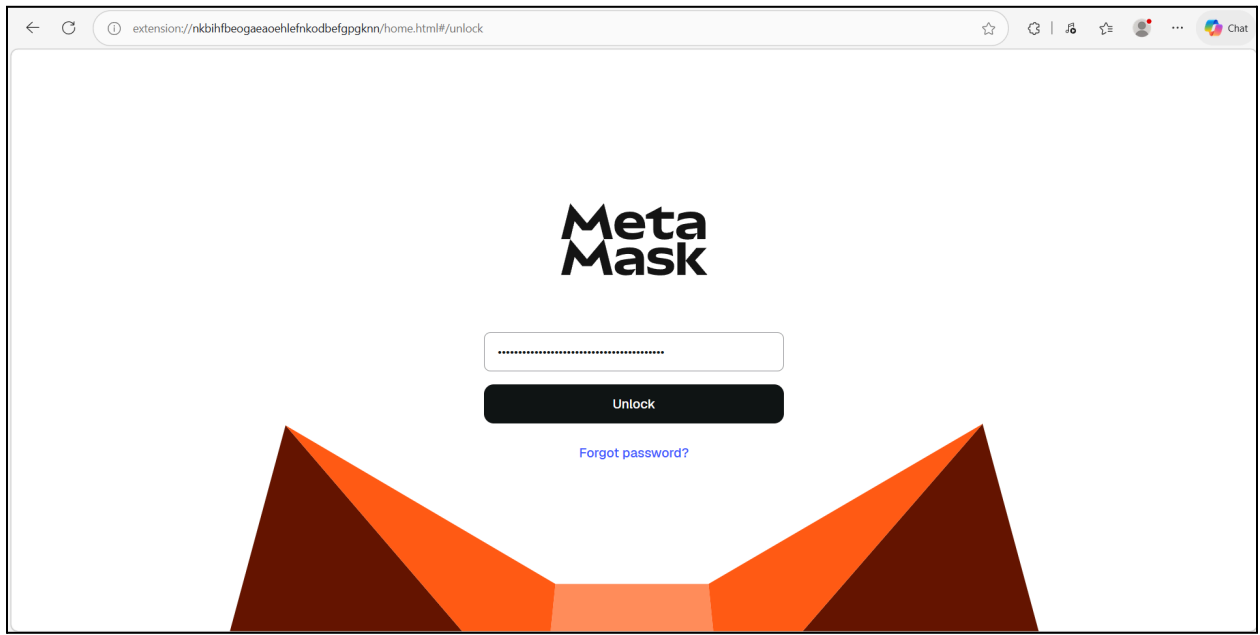
A Testnet is a blockchain network used for testing and development without using real cryptocurrency. It mimics the main Ethereum network but uses test Ether (fake ETH). Common testnets include Sepolia Testnet, Linea Testnet, and RSK Testnet.

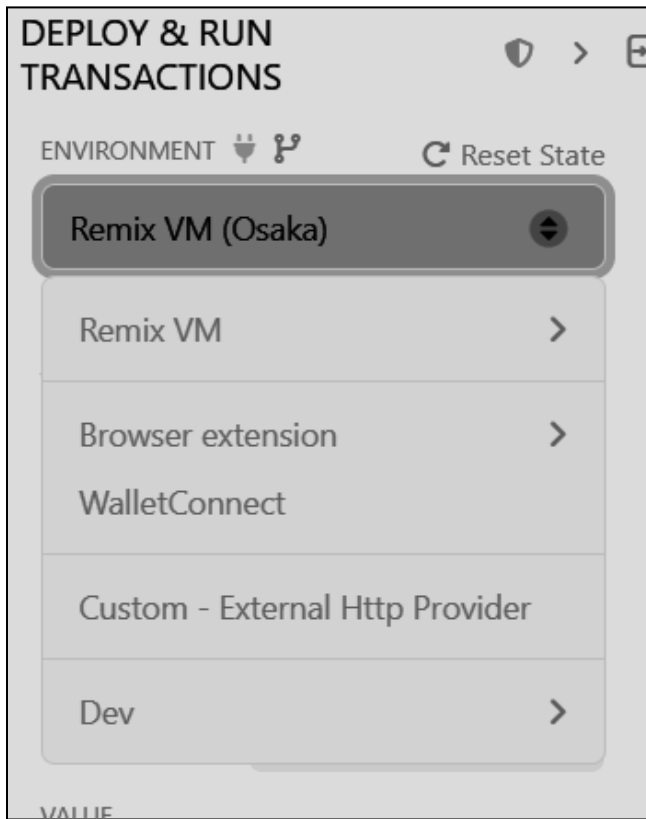
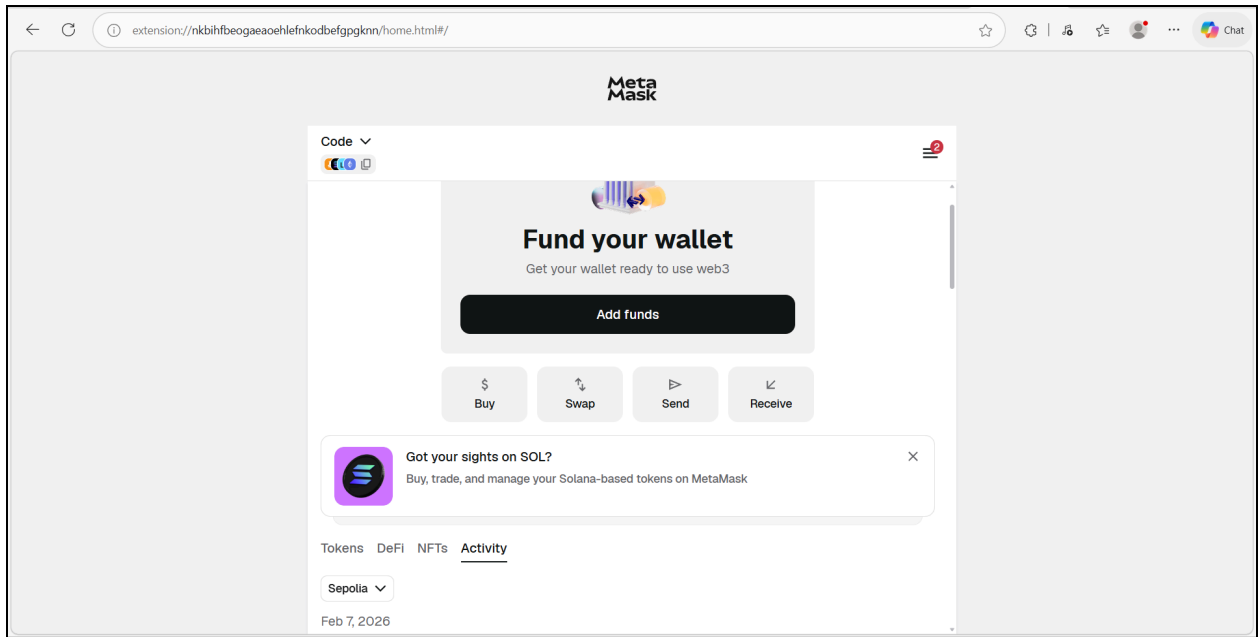
Advantages:

- No real money loss during testing
- Safe environment for developers
- Helps test smart contracts and transactions before mainnet deployment

Steps to Connect MetaMask with Remix IDE for Performing Transactions:

1. Set Up MetaMask:
 - Install the MetaMask extension on your browser.
 - Create or import a wallet account using a recovery phrase.
2. Add a Test Network:
 - Open MetaMask → Settings → Networks → Add Network.
 - Enter details for the desired Testnet (e.g., Sepolia or RSK).
 - Example RPC URL: `https://sepolia.infura.io/v3/<your_project_id>`
3. Fund the Wallet:
 - Use a faucet to receive free test Ether (e.g., 0.5 ETH/day).
 - Check the balance in MetaMask.
4. Connect MetaMask with Remix IDE:
 - Open Remix IDE.
 - In the Deploy & Run Transactions tab, choose Injected Web3 as the environment.
 - Approve connection in MetaMask.





DEPLOY & RUN
TRANSACTIONS

ENVIRONMENT

WalletConnect

Sepolia (11155111) network

ACCOUNT

0x1b4...0f039 (0.16127144765€

+ Create Smart Account

GAS LIMIT

☒ Estimated Gas

☐ Custom 3000000

VALUE

0 Wei

Sepolia test network connected to RemixIDE using metamask

Sepolia Faucet Mining

PoWFaucet Hoodi Holešky Sepolia Ephemery Devconnect 2025

Sepolia PoW Faucet



Target Address:
0x1b46C1954361A8cB698711426ffaf4f0B1b0f039

Your Mining Reward: 0.012 SepETH

Current Hashrate: 85.67 H/s

Number of Workers: 8 / 8

Minimum Claim Reward: 0.05 SepETH

Maximum Claim Reward: 2.5 SepETH

Remaining Session Time: 11h 58min

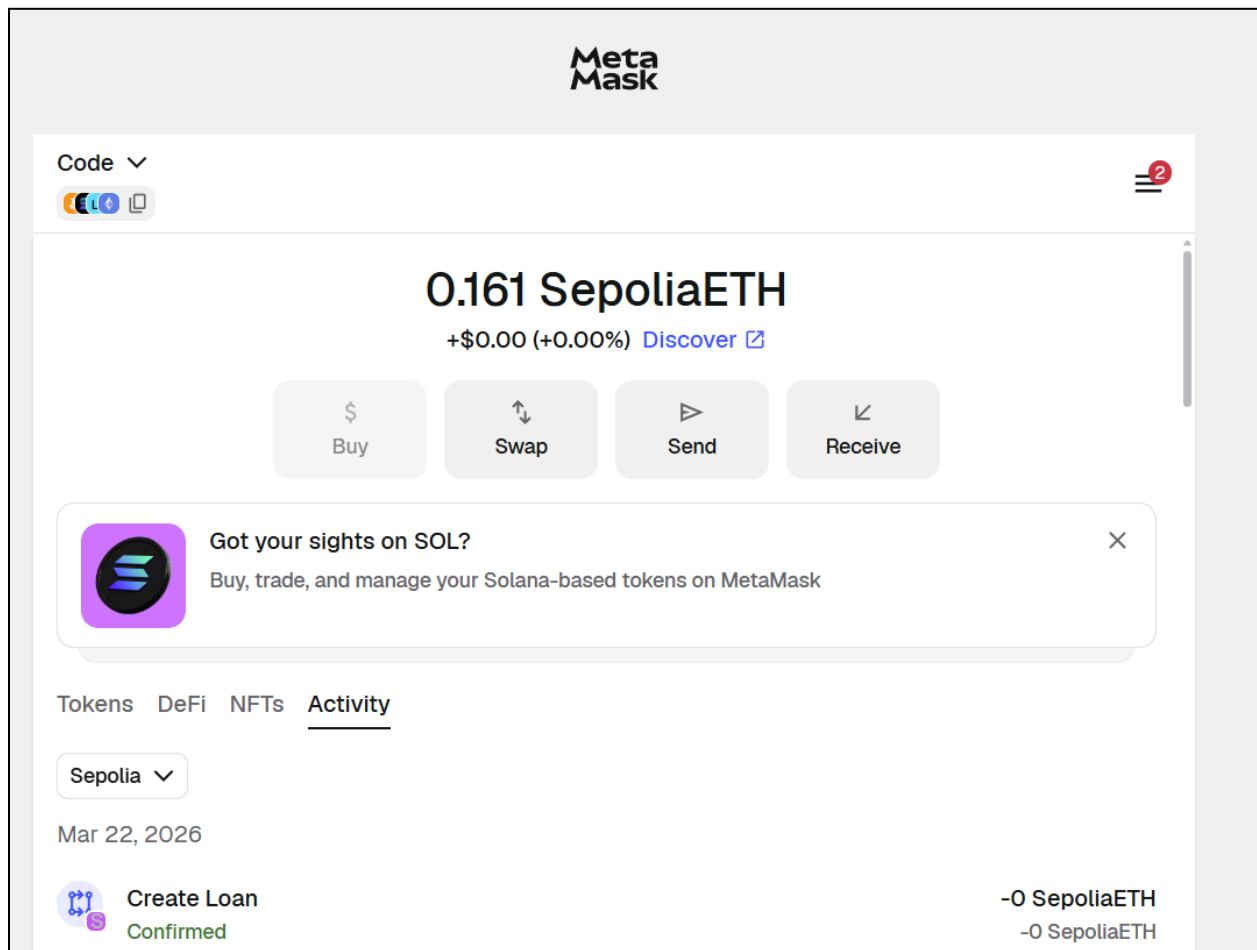
Total Shares: 1

Avg. Reward per Hour: 2.562 SepETH/h

Reward Boost: + 0%

Stop Mining

Account balance



TrustScore.sol

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.20;

contract TrustScore {

// Store trust scores for each user

mapping(address => uint256) public trustScores;

address public admin;

address public loanManagerAddress;

uint256 public initialScore = 50; // Initial score for new users

constructor() {

admin = msg.sender;

}

// Set LoanManager address (admin only)

function setLoanManagerAddress(address _loanManagerAddress) public {

require(msg.sender == admin, "Only admin");

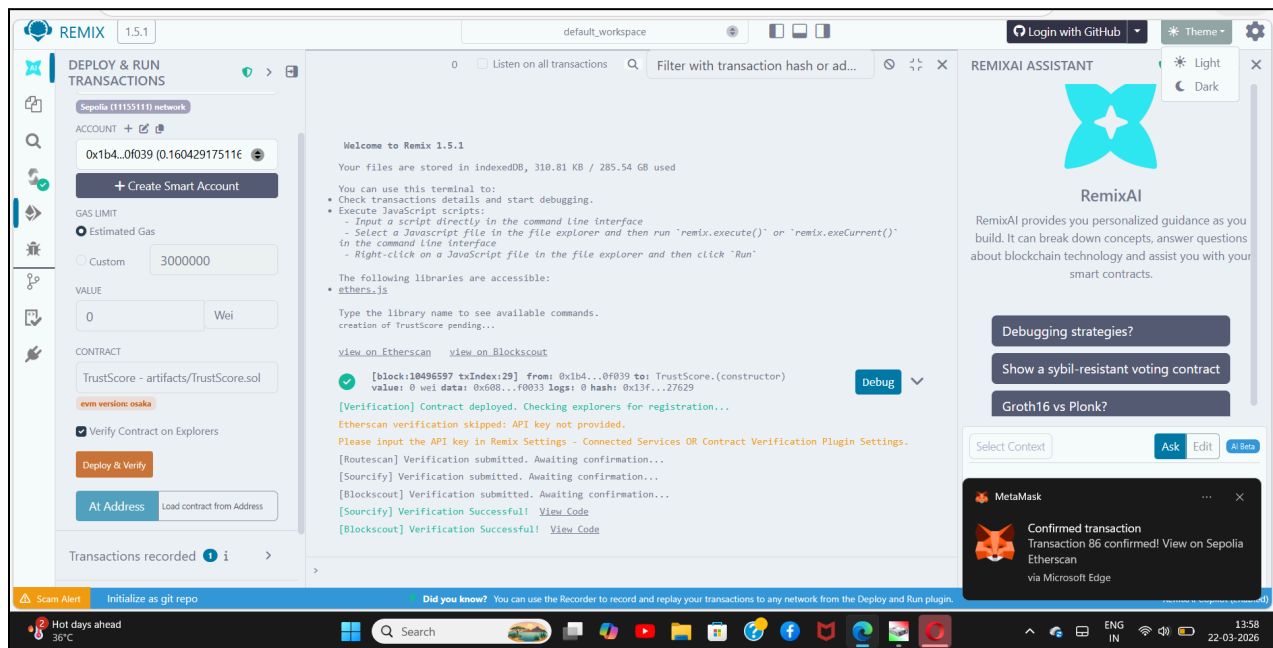
loanManagerAddress = _loanManagerAddress;

}

```
// Admin or LoanManager can update scores
function updateScore(address user, uint256 score) public {
    require(msg.sender == admin || msg.sender == loanManagerAddress, "Only admin or
LoanManager");
    require(score <= 100, "Score must be 0-100");
    trustScores[user] = score;
}

// Set initial score for new users (admin only)
function setInitialScore(uint256 _initialScore) public {
    require(msg.sender == admin, "Only admin");
    require(_initialScore <= 100, "Score must be 0-100");
    initialScore = _initialScore;
}

// Anyone can check a score - returns initialScore if not set
function getScore(address user) public view returns (uint256) {
    uint256 score = trustScores[user];
    // If score is 0, return initial score (user hasn't been explicitly set)
    return score > 0 ? score : initialScore;
}
}
```



```
view on Etherscan    view on Blockscout

[block:10496597 txIndex:29] from: 0x1b4...0f039 to: TrustScore.(constructor)
value: 0 wei data: 0x608...f0033 logs: 0 hash: 0x13f...27629 [Debug]

[Verification] Contract deployed. Checking explorers for registration...
Etherscan verification skipped: API key not provided.
Please input the API key in Remix Settings - Connected Services OR Contract Verification Plugin Settings.
[Routescan] Verification submitted. Awaiting confirmation...
[Sourcify] Verification submitted. Awaiting confirmation...
[Blockscout] Verification submitted. Awaiting confirmation...
[Sourcify] Verification Successful! View Code
[Blockscout] Verification Successful! View Code
[Routescan] Verification Successful! View Code
```

The screenshot displays the Remix IDE interface, which is divided into several panels:

- Left Panel:** Contains the 'DEPLOY & RUN TRANSACTIONS' sidebar. It shows 'Transactions recorded' (2), 'Deployed Contracts' (1), and a list of contracts. The 'TRUSTSCORE AT 0xBBD...E525' contract is selected, showing its balance (0 ETH) and a list of functions: `setLoanManager...`, `updateScore`, `admin`, `getScore`, `initialScore`, `loanManagerA...`, and `trustScores`.
- Top Panel:** Shows the 'default_workspace' and a search bar. It also includes a 'Listen on all transactions' checkbox and a 'Filter with transaction hash or ad...' input.
- Bottom Panel:** Displays the terminal output. It shows the 'Welcome to Remix 1.5.1' message, followed by instructions on how to use the terminal. The output also shows the 'TrustScore' creation process, the 'setLoanManager' function being called, and the 'initialScore' function being called. The terminal output is as follows:

```
Welcome to Remix 1.5.1

Your files are stored in indexedDB, 310.81 KB / 285.54 GB used

You can use this terminal to:
• Check transactions details and start debugging.
• Execute JavaScript scripts:
  - Input a script directly in the command line interface
  - Select a JavaScript file in the file explorer and then run "remix.execute()" or "remix.executeCurrent()"
  - In the command line interface
  - Right-click on a JavaScript file in the file explorer and then click "Run"

The following libraries are accessible:
• ethers.js

Type the library name to see available commands.
creation of TrustScore pending...

view on Etherscan    view on Blockscout

[✓] [block:10496597 txIndex:29] from: 0xb1d...0f039 to: TrustScore.(constructor)
value: 0 wei data: 0x608...f0033 logs: 0 hash: 0x13f...27629

[Verification] Contract deployed. Checking explorers for registration...
Etherscan verification skipped: API key not provided.
Please input the API key in Remix Settings - Connected Services OR Contract Verification Plugin Settings.
[Routescan] Verification submitted. Awaiting confirmation...
[Sourcify] Verification submitted. Awaiting confirmation...
[Blockscout] Verification submitted. Awaiting confirmation...
[Sourcify] Verification Successful! View Code
[Blockscout] Verification Successful! View Code
[Routescan] Verification Successful! View Code
transaction to TrustScore.setInitialScore pending ...
```
- Right Panel:** Shows the 'Transaction request' dialog. It displays 'Estimated changes' (No changes) and 'Network' (Sepolia). It also shows the 'Request from' (remix.ethereum.org) and 'Interacting with' (0xBbD4b...e5e29). The 'Network fee' is 0 s SepoliaETH, and the 'Speed' is Market ~12 sec. The dialog has 'Cancel' and 'Confirm' buttons.

Confirmation on Metamask

MetaMask



Code



Transaction request

Estimated changes ⓘ

No changes

Network

s Sepolia

Request from ⓘ

remix.ethereum.org

Interacting with ⓘ

 0xe9D99...AdC90

Network fee ⓘ

 0.0001 s SepoliaETH

Speed

Market ~12 sec

Cancel

Confirm

[Routescan] verification successful: [view code](#)



[block:10496685 txIndex:16] from: 0x1b4...0f039

to: TrustScore.setInitialScore(uint256) 0xe9D...AdC90 value: 0 wei data: 0x914...0000a

logs: 0 hash: 0xaaa...f9749

Debug



transact to TrustScore.setLoanManagerAddress pending ...

[view on Etherscan](#)

[view on Blockscout](#)

Contract details on Metmask:

Mar 22, 2026



Update Score

Confirmed

-0 SepoliaETH

-0 SepoliaETH



Contract interaction

Confirmed

-0 SepoliaETH

-0 SepoliaETH



Contract interaction

Confirmed

-0 SepoliaETH

-0 SepoliaETH



Contract deployment

Confirmed

-0 SepoliaETH

-0 SepoliaETH

Functions in contract executed :

The image shows two side-by-side panels from the Remix IDE. The left panel, titled 'Deployed Contracts', shows a contract named 'TRUSTSCORE AT 0xE9D...ADC9' with a balance of 0 ETH. It lists three functions: 'setInitialScore' (with a value of 10), 'SETLOANMANAGERADDRESS' (with a loan manager address), and 'UPDATESCORE' (with user and score parameters). The right panel, titled 'DEPLOY & RUN TRANSACTIONS', shows the execution of these functions. It includes input fields for the loan manager address, user address, and score, and buttons for 'transact', 'admin', and 'call'. The 'UPDATESCORE' function is currently selected, showing the user address '0xe9D99e9E73ca9fD72fF52f090Da8' and a score of 20.

Deployed Contracts 1

✓ TRUSTSCORE AT 0xE9D...ADC9

Balance: 0 ETH

setInitialScore 10

SETLOANMANAGERADDRESS

_loanManagerAddress: 0x1b46C1954361A8cB698711

Calldata Parameters transact

UPDATESCORE

user: 0xe9D99e9E73ca9fD72fF52f090Da8

score: 20

Calldata Parameters transact

DEPLOY & RUN TRANSACTIONS

SETLOANMANAGERADDRESS

_loanManagerAddress: 0x1b46C1954361A8cB698711

Calldata Parameters transact

UPDATESCORE

user: 0xe9D99e9E73ca9fD72fF52f090Da8

score: 20

Calldata Parameters transact

admin

GETSCORE

user: 0xe9D99e9E73ca9fD72fF52f090Da8

Calldata Parameters call

0: uint256: 20

Conclusion:

In this experiment, we successfully connected **Remix IDE** to the **Sepolia Testnet** using **MetaMask** as the wallet provider. After configuring the correct network and linking the wallet through the Injected Provider, Remix was able to interact directly with the blockchain.

Smart contracts were then compiled, deployed, and executed on the Sepolia network. The test ETH was used to pay gas fees, and all transactions were recorded on the blockchain, demonstrating a real-time decentralized execution environment.

This experiment provided practical exposure to blockchain development workflows, including wallet integration, testnet usage, contract deployment, and transaction validation, helping us understand how decentralized applications are developed and tested before mainnet deployment.